

Web Development Project

Personal Portfolio + Blog | Company Landing Page

ITEC 3020 – SUMMER

Overview

This is a single, cumulative project that you will build and expand throughout the course. It is organized into two phases: a foundation phase focused on HTML and CSS, followed by an enhancement phase where you progressively add JavaScript functionality. Each part builds directly on the previous one, so treat your codebase as a living project rather than a series of separate submissions.

This is an individual assignment. All work must be your own.

Project Options

- **Option A – Personal Portfolio + Blog**
- **Option B – Company Landing Page**

Both options require the same technical skills and cover the same components. Choose whichever option best fits your interests.

Phase 1: HTML & CSS Foundation

Objective: Build a fully responsive, multi-page website using HTML and CSS (frameworks like Bootstrap or Tailwind are permitted), showcasing your ability to structure and style content professionally.

Required Pages

1. Home Page

- **Option A:** Brief personal introduction with a welcoming message and photo(s).
- **Option B:** Company welcome message, tagline, and overview with brand visuals.

2. About Page

- **Option A:** Biography, education, skills, and interests.
 - Use lists (unordered or ordered) to organize information clearly.
- **Option B:** Company background, mission, team/people, and values.
 - Use lists (unordered or ordered) to organize information clearly.

3. Resume / Services Page

- **Option A:** Embed your resume (PDF or iframe) and include a download button.
- **Option B:** List the company's services or key offerings with icons or brief descriptions.

4. Projects / Products Page

A gallery or grid layout showcasing:

- **Option A:** Past projects with images, descriptions, and links to live demos or repositories.
- **Option B:** Products or case studies with images, descriptions, and call-to-action buttons.

5. Blog / News Page (Placeholder)

For both options: create the page structure and layout template for blog or news posts. Content will be loaded dynamically in Phase 2.

6. Contact Page

- A contact form (name, email, subject, message).
- Social media links with icons (LinkedIn, GitHub, Instagram, etc.).

Technical Requirements

HTML Elements

- **Footer:** A consistent footer on all pages including copyright information and repeated links.
- **Navigation:** A meaningful linking system to navigate between all pages.
- **Form Inputs:** A contact form with at least three different input types (e.g., text, email, textarea).
- **Variety:** Use a range of HTML elements — paragraphs, links, buttons, divs, lists, images, etc.

Custom Design

While you may use frameworks like Bootstrap or Tailwind, you are **required** to author custom CSS or meaningfully alter framework components. The final look and feel should reflect your brand. This is your opportunity to build something you can showcase as a portfolio piece.

CSS Levels

Your code must make meaningful use of CSS at **all three levels**: inline, internal, and external. Styles shared across pages belong in an external stylesheet; page-specific overrides go at the document level or inline.

Code Quality & Structure

- Well-structured, well-documented, and readable code with comments where necessary.
- Correct HTML and CSS file structure, with proper indentation throughout.
- Visual consistency: maintain a coherent color palette and layout across all pages.
- Meaningful use of CSS columns and alignment for organized, professional layouts.

Phase 2: JavaScript Enhancements

In this phase you will layer interactivity and dynamic behavior onto the website you built in Phase 1. Each part is independent but builds naturally on your existing codebase. Save all JavaScript in organized, well-commented files inside a `js/` directory (e.g., `js/theme.js`, `js/blog.js`, `js/validate.js`, `js/filter.js`).

Part 1: Theme Switcher with Persistent Preference

Objective

Implement a light/dark mode switcher that persists the user's choice across page loads using `localStorage`.

Instructions

1. Add a toggle button that appears on every page (in the navigation or header). The button label or icon should reflect the current active theme.
2. When clicked, the button switches the theme by toggling a CSS class or data attribute on the `<body>` element.
3. **Persist the preference:** save the selected theme to `localStorage` so it survives page reloads and navigation between pages.
4. On every page load, read the saved theme from `localStorage` and apply it before the page renders, to avoid a flash of the wrong theme.

Starter Code

Use and complete the following in `js/theme.js`:

```
document.addEventListener("DOMContentLoaded", function () {

    const toggleBtn = document.getElementById("theme-toggle");

    function applyTheme(theme) {
        document.body.setAttribute("data-theme", theme);
        toggleBtn.textContent = theme === "dark" ? "Switch to Light Mode" : "Switch to Dark Mode";
        // TODO: Save the selected theme to localStorage
        // Hint: localStorage.setItem("theme", theme);
    }

    function loadSavedTheme() {
        // TODO: Read the saved theme from localStorage
        // TODO: If a value exists, call applyTheme() with it
        // TODO: If nothing is saved, fall back to a default theme (e.g., "light")
        // Hint: localStorage.getItem("theme");
    }

    toggleBtn.addEventListener("click", function () {
        // TODO: Determine the current theme and toggle to the opposite one
        // Hint: check document.body.getAttribute("data-theme")
    });

    loadSavedTheme(); // Run on every page load

});
```

CSS tip: define your theme colors using CSS custom properties and use the `[data-theme]` attribute selector to switch them:

```
:root { --bg: #ffffff; --text: #111111; }
[data-theme="dark"] { --bg: #1a1a1a; --text: #f0f0f0; }
body { background-color: var(--bg); color: var(--text); }
```

Part 2: Dynamic Header & Footer

Objective

To avoid duplicating the same header and footer markup on every page, load them dynamically using JavaScript.

Instructions

5. Create separate HTML files containing only the header content (e.g., components/header.html) and footer content (e.g., components/footer.html).
6. Write JavaScript that fetches these files and injects them into a placeholder element on each page.
7. Make sure your solution works when served from a local web server (not by opening files directly in a browser, as `fetch()` requires HTTP).

Starter Code

Use and complete the following in `js/components.js`:

```
function loadComponent(selector, filePath) {
  fetch(filePath)
    .then(response => {
      if (!response.ok) throw new Error("Could not load " + filePath);
      return response.text();
    })
    .then(html => {
      document.querySelector(selector).innerHTML = html;
      // TODO: After injecting the header, re-run your theme toggle setup
      // (the button now exists in the DOM)
    })
    .catch(error => console.error(error));
}

document.addEventListener("DOMContentLoaded", function () {
  loadComponent("#header-placeholder", "components/header.html");
  loadComponent("#footer-placeholder", "components/footer.html");
});
```

Note: each page needs a placeholder element, for example: `<div id="header-placeholder"></div>`. Make sure to move your `<nav>` and `<footer>` markup into the separate component files.

Part 3: Dynamic Blog / News Section

Objective

Load and display blog or news posts dynamically from a JSON file, rather than hard-coding them in HTML.

Instructions

8. Create a JSON file to store your posts (e.g., *data/posts.json*).
9. Create or update your blog/news page (e.g., *blog.html*) with a container element to hold the posts.
10. Use and complete the JavaScript below to fetch and render the posts. Also consider the following:
 - Before displaying the posts, sort them from newest to oldest using the `date` property.
 - Display the publication date in a reader-friendly format (e.g., "June 15, 2025") using JavaScript's `Date` object.
 - Add a "Latest Post" badge beside the most recent post.
11. Style each rendered post card to match the rest of your site.

Sample posts.json structure

```
[
  {
    "id": 1,
    "title": "My Post Title",
    "date": "2025-06-01",
    "summary": "A short description of the post.",
    "category": "Tutorial",
    "content": "Full post body text goes here."
  }
]
```

Starter Code (js/blog.js)

```
document.addEventListener("DOMContentLoaded", function () {
  const blogList = document.getElementById("blog-list");

  fetch("data/posts.json")
    .then(response => response.json())
    .then(posts => {
      posts.forEach(post => {
        const postElement = document.createElement("div");
        postElement.classList.add("post-card");

        // TODO: Populate postElement with the post data
        // Suggested structure:
        //   <h2> for post.title
        //   <p class="post-meta"> for post.date and post.category
        //   <p> for post.summary
        //   A "Read More" button or link (can toggle full content visibility)

        blogList.appendChild(postElement);
      });
    })
    .catch(error => console.error("Error loading posts:", error));
});
```

Part 4: Contact Form Validation

Objective

Validate the contact form on the client side before submission, providing clear and helpful feedback to the user.

Instructions

12. Intercept the form's submit event and prevent the default submission.
13. Validate each required field: name (not empty), email (valid format), message (minimum length of 20 characters).
14. Display an inline error message beneath each invalid field. Clear errors when the user corrects their input.
15. If all fields pass validation, show a success message in place of the form (or simulate a submission).

Starter Code (js/validate.js)

```
document.addEventListener("DOMContentLoaded", function () {
  const form = document.getElementById("contact-form");

  function showError(fieldId, message) {
    const field = document.getElementById(fieldId);
    // TODO: Find or create an error element next to the field
    // TODO: Set its text to message and make it visible
    // Hint: use field.nextElementSibling or a dedicated <span class="error-msg">
  }

  function clearError(fieldId) {
    // TODO: Hide or clear the error message for this field
  }

  function validateEmail(email) {
    // TODO: Return true if email matches a valid format, false otherwise
    // Hint: /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)
  }

  form.addEventListener("submit", function (e) {
    e.preventDefault();
    let isValid = true;

    const name = document.getElementById("name").value.trim();
    const email = document.getElementById("email").value.trim();
    const message = document.getElementById("message").value.trim();

    // TODO: Validate name - show error if empty
    // TODO: Validate email - show error if empty or invalid format
    // TODO: Validate message - show error if fewer than 20 characters
    // TODO: If isValid is true, show a success message
  });

  // Bonus: clear errors as the user types
  ["name", "email", "message"].forEach(function (id) {
    document.getElementById(id).addEventListener("input", function () {
      clearError(id);
    });
  });
});
```

Part 5: Projects / Blog Filter

Objective

Add a live search or category filter to your Projects/Products page (Option A/B) and your Blog/News page, so users can find content without reloading the page.

Instructions

16. Add a text input (search box) or a set of category buttons above the content grid.
17. As the user types or selects a category, instantly show only the matching cards and hide the rest, no page reload.
18. Match against at least two fields (e.g., title *and* category or description).
19. If no results match, display a friendly "No results found" message.

Starter Code (js/filter.js)

```
document.addEventListener("DOMContentLoaded", function () {
  const filterInput = document.getElementById("filter-input");
  const noResults = document.getElementById("no-results");

  // Adjust the selector to match your card markup
  const cards = document.querySelectorAll(".project-card");

  filterInput.addEventListener("input", function () {
    const query = this.value.toLowerCase().trim();
    let visibleCount = 0;

    cards.forEach(function (card) {
      // TODO: Read the title and category (or description) from the card
      // Hint: card.querySelector(".card-title").textContent.toLowerCase()

      // TODO: Show the card if query matches title OR category/description
      // TODO: Hide it otherwise
      // Hint: card.style.display = "none" or "" (empty string to restore)

      // TODO: Increment visibleCount for each visible card
    });

    // TODO: Show noResults element if visibleCount === 0, hide it otherwise
  });
});
```

Tip for the Blog page: since posts are dynamically generated in Part 3, run the filter setup *after* the posts have been appended to the DOM, or use event delegation to handle dynamically added elements.

Part 6: Blog Documentation

Objective

Document your development process by writing blog posts that explain how you implemented the JavaScript features above.

Instructions

20. Add at least four blog posts to your posts.json file (one per Part: 1, 3, 4, and 5).
21. Each post should explain: what the feature does, how you approached it, any challenges you encountered, and what you learned.
 - When generating each blog post, use the properties of the JavaScript object (title, date, category, summary, and content) rather than hard-coding any values.
22. Posts should be written in your own words and reflect your actual development experience.
23. The posts will be displayed automatically via the dynamic blog section you built in Part 3.

Publishing on GitHub

- Follow the links shared on eClass (Week 4) to publish your website on GitHub Pages. Name your main file index.html for easy deployment.
- You may use another version control platform or hosting service as long as you can provide links to both the repository and the live site.
- **Commit history matters:** regular, meaningful commits demonstrate good version control practice and will positively impact your grade. Commit at logical milestones, not just at the end.

Submission

Submit a ZIP file and a PDF document containing:

ZIP file:

24. All HTML, CSS, and JavaScript files.
25. The data/ folder (including posts.json) and the components/ folder.
26. All assets (images, icons, fonts, etc.).

PDF document:

27. Your full name and student number.
28. Link to your GitHub (or other Git) repository.
29. Link to your published website.

Grading Criteria

Phase 1: HTML & CSS Foundation

- Required pages present and complete with appropriate content for the chosen option.
- Correct and varied use of HTML elements, including semantic structure.
- Meaningful use of CSS at all three levels (inline, internal, external).
- Responsive layout and visual consistency across pages.
- Custom CSS design choices beyond default framework styles.
- Code quality: readability, indentation, and comments.

Phase 2: JavaScript Enhancements

Part 1 – Theme Switcher: toggle works correctly; preference persists across pages and reloads via localStorage.

Part 2 – Dynamic Components: header and footer load correctly without duplication.

Part 3 – Dynamic Blog: posts are fetched from JSON and rendered with appropriate styling.

Part 4 – Form Validation: all fields validated with clear inline error messages; success state handled.

Part 5 – Filter: live filtering works on title and at least one other field; no-results state handled.

Part 6 – Blog Posts: at least four posts present, each clearly explaining an implemented feature.

Overall

- Version control: regular, meaningful commit history on GitHub.
- Code quality, organization, and readability across all files.
- Cohesion: the site feels like one unified project across both phases.